The Printed Rows of a Translation - Layout Guidelines

▼ Contents

1. What this is about 1
2. The layout module 2
3. The tool 5
4. How to work 9
5. What a good break is 10
   Lessons 11

> What a layout session needs to lay out the ancient-text translations - on top of the general guidelines, _RTL-EDITOR/layout-guidelines/claude-layout-guidelines.md, its system prompt. The logic it works with is beside this file: numbered-lines.ts (what a numbered line is, and which run on), rows.ts (what a break costs, and the rows that cost least), translation-layout.ts (what a translation's layout module calls), rows-tool.ts (the tool). **This is a living document**: a session that learns something reusable adds it to §5's *Lessons*, in one line.

1. What this is about

Every `translation.rtl.md` under `שונות/תרגומי-טקסטים-קדומים/` is written a line of the tablet per line of the file, each line opening with its number:

```
**1** כאשר אָנוּ הנעלה,
**2** מלך הָאֲנוּנָכִי,
**3** וְאֶנְלִיל,
```

A tablet line is often two or three words, so a page with a line per row is a narrow column down one edge of the sheet: hard to read and wasteful to print. So the HTML pages and the PDFs run consecutive numbered lines together into **rows**. Each line is kept whole, a thin grey bar stands between two lines on one row, and only a row's first number is shown, in the margin:

```
 1  של אֲאָה | הבן הראשון | לַמַּרְדּוּךְ, | גורלות הארץ, | הגוזר | והארץ, | אדון השמים |
11  בשמה הנעלה קראו | את בָּבֶל | הגדילוהו, | בְּאֵיגִיגִי | גזרו לו, | על כל האנשים | יל
18  כוננו לו | מושתתים, | יסודותיה | והארץ, | אשר כשמים | מלכות נצח | בתוכה | וה,
```

Without a layout, the page fills each row until the width runs out and breaks there, which is often mid-phrase ("הבן הראשון |" at the end of a row, "שֶל אֵאָה," at the start of the next). **Your job is to decide where the rows break**, so that every row is:

1. **as long as the printed A4 row allows**, and
2. **never longer**: a row that wraps on paper is the one outcome that is always wrong, and
3. **broken where the text itself pauses.**

You decide it once per translation, in its own layout module, `translation.rtl.layout.ts` (§2) - the only file you write (the general guidelines, §2).

2. The layout module

A list of breaks - the form a hand-made layout takes (Atrahasis):

```
import type { FileLayout, LayoutOfFile } from "../../../../_RTL-EDITOR/src/html/
import { translationLayout } from "../../layout/translation-layout.ts";

// The MD5 of translation.rtl.md as these rows were made for it.
const SOURCE_MD5 = "a3dbf6827c6e07e5763fd11adf7cbd45";
// The row starts of translation.rtl.md - 449, by the rows tool.
const ROW_STARTS: readonly string[] = [
    "1", "11", "18", "27", "37", "47", "50", ...
];
// Rows a tool made and no session has read yet (propose --fix) - check fails wh
const REVIEW: readonly string[] = [];

// The return type is spelled out so that tsc catches a misspelled entry, which
const layout: LayoutOfFile = (file, above): FileLayout => ({
    sourceMd5: SOURCE_MD5,
    ...translationLayout(file, above, { rowStart: ROW_STARTS, review: REVIEW }),
});
export default layout;
```

`translationLayout()` (`layout/translation-layout.ts`) is the translations' own logic: it finds the file's numbered lines and runs, decides the rows as it is told - `rowStart`, or `rowBreak` below - and hands the editor what it knows, line groups (the general guidelines, §1).

(`template <file>` prints this, with the right relative path to the contract, the file's MD5 and its current rows.)

`ROW_STARTS` lists, in the order of the file, **the lines that start a row**, by their label: the bold text the line opens with (`"1"`, `"A.i.5"`, `"o iii 12"`, `"1123'"`). The numbering often starts over (each tablet of Atrahasis has its own line 1), so a repeated label is named by its occurrence: `"1#2"` is the second line labelled `1` in the file, `"1#3"` the third.

Every line not listed continues the row of the line before it. There is one exception, which you cannot override: where the numbering itself breaks, a new row starts anyway. That happens at a new column (`A.i.30` → `A.ii.1`), a gap (`5` → `9`), a line of prose, or a heading. A stretch of consecutive numbering is a **run**; the tool names the first line of every run in the file, so the list reads complete.

What does *not* break the numbering, and so flows with its neighbours:

- **A manuscript's extra line**: a letter after the number, Hebrew or lowercase Latin. `363` → `363 364` → … → `בג363` → `א`, and `E 78` → `E 78a` → `E 79`.
- **A range**: `65` → `66–67` → `68`.

A few labelled lines are in no run at all, such as a note (`I 7a–i`) or a stretch of line beginnings (`iv 39–47`). Each is a row of its own and needs no name.

- To **move a break**, replace a name: `"10"` → `"11"` moves line 10 up to the end of the row above.
- To **split a row**, add a name. To **join two rows**, remove the second one's name.
- A name that matches no line is ignored. The tools report it as **stale** (it happens when the translation is edited after the layout was made). Fix it.

A rule - the form to prefer (the general guidelines, §2). `template <file> --rules` prints this, ready to fill in:

```
import type { FileLayout, LayoutOfFile } from "../../../../_RTL-EDITOR/src/html/
import { translationLayout, type BreakContext, type BreakCost } from "../../layo

const SOURCE_MD5 = "…";

/** Where a row must not end: after a line whose text ends so - named by its wor
const KEEP_WITH_NEXT: readonly string[] = ["ואיש נגף והמית" ,"הבן הראשון —"];
/** Where a row must end: after a line whose text ends so. */
const BREAK_AFTER: readonly string[] = ["תקנת המלך."];

function rowBreak({ line, cost }: BreakContext): BreakCost {
    if (KEEP_WITH_NEXT.some(words => line.text.endsWith(words))) return "never";
    if (BREAK_AFTER.some(words => line.text.endsWith(words))) return "always";
    return cost;
}

const layout: LayoutOfFile = (file, above): FileLayout => ({
    sourceMd5: SOURCE_MD5,
    ...translationLayout(file, above, { rowBreak, repeatedPassages: "alike" }),
});
export default layout;
```

- **How it works.** For every run, the mirror asks `rowBreak` what ending a row after each line (but the last) costs, and lays the run out as `propose` does: rows as full as they can be (a row's emptiness, squared, costs 100 × it), plus what its breaks cost. `cost` in the context is the default - `breakPenalty()`: 0 after a sentence, 4 after a clause, 6 after a dash, 12 mid-phrase, +4 before a ו. Return it to keep the default; a number of your own (e.g. `cost + 10` to discourage, `cost - 3` to encourage); `"always"` to end the row there; `"never"` to keep the line on one row with the next (if they cannot share one at all, they will not).
- **Rules beyond punctuation** are welcome: a formula that opens a speech (`next.text.startsWith("פיו פתח")` → `"always"`), a line that is all lacuna (`/^[\[\]…\s.]*$/.test(line.text)` → a cheap break), a law's protasis and apodosis. `measure.line(line)` gives a line's width, in percent of a row, should a rule need it.
- **Units** - sentences, couplets - a rule may keep whole. The run is parsed into units, and the context says the unit of each line (`unit`, by place in the run) and whether the line ends it (`endsUnit`). By default a unit is a sentence: up to a line ending with `.` `!` `:`, a real `?` (before a closing quote) or the close of a direct speech (`"`) - a `;` does not end one. To keep units whole: `rowBreak: ({ endsUnit, cost }) => endsUnit ? cost : cost + 150` (a break inside a unit has to outweigh the fill: a half-full row costs ~25). Units of your own: `unitEnd`, asked about each line (`UnitContext`) whether a unit ends with it - `({ line }) => /[.!:;]$/.test(line.text)` to count `;` too.

- **Repeated passages**: `repeatedPassages: "alike"` breaks a passage that repeats an earlier one word for word (the context's `file.echoes`, vowel points and punctuation aside) exactly where its first occurrence breaks - so lay the first occurrence out well, and the rest follow. An exception decided outright (`"always"`, `"never"`) still stands in a repetition, so a repeat that should differ can be named on its own. `"similar"` takes nearly the same words too (`file.similarEchoes`): a herald's orders said again with a letter, a name or a person changed (`צוו,` / `ויקראו הכרוזים` / `[צו]ו, וקראו [הכרוזים]` , `את נמתר – בקשו` / `את אדד – בקשו`), an order and its carrying out, a speech formula with another speaker. Two neighbouring lines repeat two said before when their letters are mostly the same (numbered-lines.ts's `lineSimilarity()`, at least 0.6, and one of the two 0.75). It finds more than you may want in a law code, where every law resembles the one before - read what it did with `compare`.
- **A minimum fill**: `minimumFill: 45` keeps every row but a run's last at least 45% full wherever the next line would still fit on it - so a lone short line (a lacuna, `[...]`) joins a row rather than taking one of its own. It outweighs every cost but `"never"`; an `"always"` still ends the row, however empty.
- **See what the rule does** with `report <file> --text` (it shows the rows the mirror will print), and **what it changed** with `compare <file> --with=<other module>` - against the previous version of your module, or against a hand-made list, place by place with each side's rows. Its count of places is the measure of how far two layouts are apart.
- A list (`rowStart`) overrides a rule wherever both are given - the folder's default rule, too.

`rowStart` may instead be a function, asked about every line of a run but its first, in order, whether it starts a new row (`RowContext`, in `translation-layout.ts`). It is given the row so far, `row.fits(line)`, and `measure` - widths as a percentage of the row, computed from the very fonts the PDF is printed in (`check` reports how far they are from Chromium's: a few hundredths of a percent). For instance, a translation whose decisions were all "break at a sentence's end once the row is half full, or when the line does not fit":

```
rowStart: ({ previous, row, line }) => !row.fits(line) || (/[.;:!]$/.test(previc
```

Try a module before replacing the file's own: write it anywhere, and give it to any command with `--layout=<path>`.

3. The tool

From the session's folder (or anywhere under `_RTL-EDITOR/`):

```
npm run rows -- <command> <path/to/translation.rtl.md> [options]
```

(npm passes the options on only after the `--`, and runs the tool in `_RTL-EDITOR/`: give the path from the repository root. It is `layout/rows-tool.ts`, beside these guidelines: `node rows-tool.ts …` there is the same, and takes a path from there too.)

It renders the file exactly as the mirror does - with the layout module, the fonts and the print settings - and lays it out in the same Chromium, at the same width, as the PDF printer. So **its measurements are the PDF's.** It takes a second or two per file. It reads the module and never writes it (the general guidelines, §2).

Command	What it does
`status [folder]`	Every file under the folder (the whole tree by default) that should have a layout of its own (`output: "laid-out"`), or has one, with the state of its rows: `no layout`, `NEEDS WORK` (rows that overflow, stale names, rows to review, the file changed), `ok`.
`propose <file>`	The rows a dynamic program would choose. They are as full as possible, and it prefers to break after `.` `;` `:` `!`, then after `,` `?` `…`, then after a dash, and least of all after a line with no punctuation. A `?` counts as a sentence end only before a closing quote, since here it usually marks an uncertain reading. It also avoids breaking before a line that opens with ı. Prints the rows, and then the row starts as `ROW_STARTS` / `REVIEW` for the module (or into `--out=<path>`); `--plain` ignores punctuation and only fills.
`propose <file> --fix`	Repairs a layout: keeps **every** break it has, and only splits the rows that overflow now. The rows it made are listed in `REVIEW` until a session has read them. `check` fails while any is left.
`report <file> --text`	The rows of the current layout, one per line: the label range, how full the row is (`%` of the column), the flags, and the text of its lines separated by `|`. A labelled line in no run is shown in its place, marked `·`, so a run boundary is seen in context.
`report <file> --widths`	The same with every line on its own line, with its width as a `%` of the column. A run's header gives the width of the gap between two lines. Plan breaks with this: a row fits when its lines plus its gaps add up to under 100%.
`report <file> --run=3,7-9`	Only those runs, as the report numbers them. Combines with `--text` / `--widths`.
`report <file> --review`	Only the runs holding a row marked `REVIEW`.
`try <file> --run=N --move=7>8`	Run N with its current breaks changed: `--move=from>to` moves a break, `--add=` adds one, `--drop=` removes one (each takes a comma-separated list). Every other run stays as the layout has it. It prints that run's rows, and then the whole file's `ROW_STARTS` / `REVIEW` with the change made - and the run's `REVIEW` marks cleared, since you have now read it. **The way to edit a run**: put the constants it prints into the module.
`try <file> --run=N --starts=41,43,45`	The same, with the run's row starts given whole: every start except the run's first line, which is implied. `report` prints each run's current list under its header, ready to copy and edit.

Command	What it does
`check <file>`	Prints the summary line, type-checks the module, and says how far the widths a layout computes are from Chromium's. Exits 0 only if the module is sound, it has rows, no row overflows, no name is stale and no row is marked `REVIEW`.
`resolve <file>`	The file's layout as the mirror gets it, with every function in it called - as JSON.
`template <file> [--rules]`	A `<name>.layout.ts` to start the file's layout with: its MD5 and current rows as a list, or (`--rules`) a rule with room for exceptions (§2).
`compare <file> --with=<module>`	Where the rows of the file's layout (or of `--layout=`) differ from those of another module, place by place - each side's rows, with their text - and how many places differ, in how many runs. A place is a stretch between two lines both sides start a row with: one run may hold several, each a decision of its own.
`md5 <file>`	The file's MD5 - what `SOURCE_MD5` must be.
`preview <file> --pages=4-6,12 --out=<folder>`	Prints the PDF as the mirror will, says how many pages it has, and saves those pages as PNGs. Read them to see the result.

Every command takes `--layout=<module>`, to use that module instead of the file's own.

The flags in `report`:

- **`OVERFLOWS`**: the row is wider than the column and wraps on paper. Always fix it.
- **`TOO-WIDE-ALONE`**: a single line wider than a whole row. Nothing can be done about it (the line wraps inside itself). Leave it on a row of its own, and mention it in your report.
- **`room`**: the first line of the next row would still fit on this one. Not an error, but a hint: either the break is there for a reason (a sentence ends), or the row can take one more line.
- **`REVIEW`**: made by `propose --fix` and not yet read.

`report` prints a run (a stretch of consecutive numbering) as a block headed `run N: lines X .. Y`. A row is shown the same way, `5 .. 9`, because a label may hold a dash of its own (`66–67`). In a long file (Hammurabi has ~550 rows), work through it run by run with `--run`.

A label with a space in it is quoted as a whole option: `--move="G 28>G 29"`.

The summary line gives the number of rows next to **the fewest possible**: every row filled until the next line no longer fits, whatever the text says. It gives the average fill twice, without and with the last row of each run. Judge the fill against the fewest possible, not against a fixed

percentage. A law code of short laws has many short runs, and two thirds may be all there is (Eshnunna's rows are about as few as possible).

4. How to work

1. **Ask which texts**, unless the user already said. If they say "all", or nothing, run `status` and suggest an order: files with `NEEDS WORK` first (their printed pages are wrong), then `no layout`, the shortest first.
2. Per file:
    1. Read the file's head (its introduction says what kind of text it is: verse, a law code, a list of proverbs, letters). The kind of text decides what a "pause" is (§5).
    2. No layout of its own yet: the folder's default rule lays it out already - `report <file> --text` shows its rows. Start your module from a rule: `template <file> --rules > <folder>/translation.rtl.layout.ts`.
    3. **Read every row.** This is the part no tool can do. For each row ask: does it end where a Hebrew reader would pause? Does the next row start with something that can start a sentence? Where it does not, find the rule or the exception that says why (§2, §5): a keep-together named by its words, a cost for a kind of line. Keep a copy of the module before each change, and `compare --with=` it to see everything the change moved - a rule moves more than the break you meant. `report --widths --run=N` gives the numbers to plan with. If a text needs more exceptions than rules, turn to a list: `propose`, then `try` run by run, putting what it prints into the module.
    4. `check <file>` until it exits 0. Then look at the summary. A layout with 10–20% more rows than the fewest possible is normal for verse, where couplets are kept whole (Atrahasis: 449 rows against the fewest possible 380). Don't chase fill beyond that.
    5. `preview` a few pages and look at the PNGs: the first one, one from the middle, and one from a stretch the report shows as dense (many short lines, or many variants). `preview` says how many pages there are.
3. **Report to the user** per file (the general guidelines, §2): how many rows, how full, what you decided and why where it was not obvious, every `TOO-WIDE-ALONE` line, and anything in the translation that looked wrong to you (you did not change it).

When a translation is edited later, `layout-status` lists it as `OBSOLETE` (and rows a tool split as `ATTENTION`). If lines were added, removed or renumbered, the translation session that edited it has already run `propose <file> --fix` itself, put its constants (`SOURCE_MD5` among them) into the module, and said so in its report. That keeps every decision already made, splits only the rows that overflow now, and lists the rows it made in `REVIEW`; `status` shows the file as `NEEDS WORK` until a layout session has read them. Then:

1. If `layout-status --check` says the rows still fit, put in the new `SOURCE_MD5` and you are done. If rows overflow (no `--fix` was run), run `propose <file> --fix` yourself, and put its constants in.

2. Read the `REVIEW` rows with `report <file> --review --text`, and settle each run with `try`, even when you change nothing, to clear its marks - putting the constants it prints into the module each time.
3. Fix any stale names (`check` lists them).

Don't re-propose the whole file over decisions already made.

5. What a good break is

In order of strength, a row should end:

1. **At the end of a sentence** (`.` `;` `:` `!`, or the `?` of a real question; most `?` here mark an uncertain reading), or at the end of direct speech.
2. **At the end of a clause or an epithet** (`,`). In the long chains of epithets of a royal prologue ("המשיב את אֶרְידוּ למקומה, המזכך את טהרת אֵאַבְּזוּ;"), each epithet is a unit, however many tablet lines it spans.
3. **Never inside a phrase.** Don't separate a construct chain or a "של" phrase from its head ("הבן הראשון | של אֵאָה"), a preposition or "את" from its noun, a verb from its object, or "אשר" from the clause it opens, when any other break is available.

And beyond the single break:

- **Verse**: keep a parallel couplet (A ‖ B) on one row where it fits, and prefer starting a row at the start of a couplet or sentence. A repeated formula ("פיו פתח ואמר, אל ... דבר") is a unit.
- **Laws**: a law (`§`) usually starts with its own heading, so the runs are short already, often one or two rows. Judge a law by its rows together, not by its first row's fill. Prefer breaking between the protasis ("אם איש ...") and the apodosis ("... יומת"). That break often falls inside a tablet line, because the Hebrew keeps the Akkadian order ("... ואיש נגף והמית – בעל השור"). Then, in order:
  - never leave a lone verb or a one-word row ("ישקול." at 7%);
  - prefer a comma inside the protasis over splitting the apodosis;
  - lay parallel laws out alike (the goring ox and the mad dog).
- **Broken text** (`[...]`, `...`): a line that is almost all lacuna is a natural place for a break on either side of it. Don't let a row end with an open bracket that the next row closes, when avoidable.
- **Fill still matters.** A row of two words followed by a full row usually means the break is in the wrong place. Prefer moving a break by one or two lines over leaving a row half empty, unless the half-empty row ends a sentence and the next one begins a new unit.
- A row that ends a sentence early is better than a full row that ends mid-phrase. A full row that ends at a comma is better than a half-empty one that ends at a full stop. Use judgement between those.

- **A split couplet is worse than a short row, down to about 50% full.** Below that, prefer filling the row, unless the short row is a unit of its own (a speech formula, a heading-like line, the end of a passage).
- **A repeated passage is laid out the same way each time it recurs**: a herald's orders repeated word for word, a guard formula, a refrain. The reader sees the repetition in the shape of the rows too.
- **Variant lines** (`363{נוסח אשורי}, א ...`) are wide and often come in pairs or sets. Keep a variant with the lines it belongs to when it fits. Read which lines those are: it may be the line *before* it (an alternative wording) or the lines *after* it (167ב–א introduce 168–169). A set of variants that together replace a passage may take rows of their own. A variant that is all lacuna (`19א [...]`) belongs to nothing in particular, and is a free place for a break.

Lessons

(Add one line per lesson: the date, the text, what you learned.)

- 2026-10-21, Atrahasis (the pilot): most of the proposal's faults were split couplets. Expect to move breaks in about half the runs of a verse text, and to end around 77% fill. `?` marks uncertain readings in these files, not questions.
- 2026-10-21, Atrahasis (the second pilot): the repeated-passage rule drove about a third of the changes. Lay out the first occurrence well, then copy its shape to the rest.
- 2026-10-21, Eshnunna (the second pilot): in a law code, judge a law by its rows together. The fill drops when weight moves to a law's last row, though the row count is the same.
- 2026-10-23, Atrahasis and Eshnunna (re-read for David CLM): in the wider font two couplets rarely fit on one row (105–110%), so a verse text that keeps its couplets whole ends ~18% above the fewest rows, not 10%. Where the lines are a sequence of actions rather than parallel pairs, three lines broken at a sentence end beat two half rows. A law code's short runs still reach the fewest rows.
- 2026-10-24, Eshnunna (rules instead of a list): a run's last row is free, so in short laws the default fills the first row and strands the law's tail on the second, mid-phrase. Raising the cost of a line that ends with no punctuation (+40 - the Hebrew keeps the verb last, so such a line is inside its clause) recovered 4 of the hand list's 11 better runs at once; four small rules and one named exception (a quotation kept whole, a break before a lacuna as cheap as a comma, the warned-owner laws alike) did the rest - 95 rows, the same as the hand list, with no list at all.

- 2026-10-26, Atrahasis (rules instead of a list - kept the list): verse did not go the way of Eshnunna. The fill term (100 × emptiness²) charges ~20-30 for every half-full couplet row, so a rule has to make a break *inside* a couplet cost ~150 before couplets survive. Line parity does not mark couplets (one tercet or range shifts it), so the rule has to parse the run into units itself: sentences, from `./!/:` to the next one, with `;` inside a unit, a real `?` only after a question word, and the end of direct speech a strong break. Even then it plateaued at ~77% of the hand list's row starts and 60 places that differ, 36 of them worse - mostly tercets and triads the hand kept whole, a passage repeated with small changes in wording (which `alike` does not see), and one-line lacuna rows. Fixing them took about one exception per place, and the exceptions got in each other's way. In verse the decision is the reader's sense of parallelism, which punctuation does not show; keep a careful hand list.
- 2026-10-27, Atrahasis (rules instead of a list - switched): with `unitEnd` it worked. A unit parser of its own (sentences; a real `?` only in a sentence with a question word; a speech opening ends the unit before it; two one-line sentences in a row are one couplet, unless a speech edge is between them or the second's letters are clearly closer to the line after it) plus parity inside a unit (+60 after an even line, +150 after an odd one) did most of it; speech kept to its own rows (always break after a speech closes, unless a formula or another speech follows) did much of the rest. 19 exceptions named by words - mostly parallel pairs no punctuation shows - left 41 places differing from the hand list, 24 judged better (a sentence the hand split, a repetition the hand laid out unlike its first occurrence), 17 the same, none worse; 460 rows against 449. A `"never"` loses to a repeated passage's forced break: name the first occurrence too.
- 2026-10-27, Sargon and Bird and Fish (tool pitfalls): `rowBreak`'s `line.text` is the raw Markdown (`...`, `[^n]`), unlike `report --text` - strip the markup or name exceptions by words with none in them. Stripping vowel points with `[־]` also strips the maqaf (U+05BE), so a name with ־ silently matches nothing: use `[־]`. Run `report` first and look at "lines in no run": a label with two numbers (`A1 (37)`, Ziusudra) makes no runs at all.
- 2026-10-27, eleven short texts (rules, no lists): the kind of text names its rule. An omen list (Advice to a Prince): start a row at `(אִם)` and at the god who acts (the apodosis). Instructions (the Farmer): a line ending in a dash leads to its order (+100 after it). Letters: the address formula ("אל X אשר זה אמור: Y אומר:") an `"always"` row of its own, ~10% more rows, the same shape in every letter. A dialogue (Pessimism): a unit also ends before a line that opens a speech or a quotation. Gnomic verse where every line ends with `;` (Counsels of Wisdom): sentence units and a flat +40 inside them - parity misfires. A hymn of strict couplets (Shamash): count couplets from each section's start and name the few tercets and lone lines - 8 names for 200 lines. Short narrative verse (Ishtar): +30 inside a sentence, always break at a speech's close, ~8 keep-togethers. All ended 1-13% above the fewest rows.

- 2026-10-27, the other 31 texts (rules, no lists, 0-16% above the fewest rows): **Laws**: Eshnunna's base (+30-40 after a line with no pause, a cheap break after a lacuna) and a *case* rule - a row starts at "(ו)אם" after a sentence end (–15, or `"always"`), +30 for leaving a case's opening line at a row's end (MAL A, Hammurabi). A royal prologue's epithet chains need the exceptions; the laws do not (Ur-Namma, Hammurabi: 19 names for 3600 lines). A treaty's curses: a cheap break at "ככה" (Esarhaddon). **Verse**: strict couplets - count from each section's start and name the tercets and lone lines (Enuma Elish: 4 names; Ludlul: 26). Where punctuation does not mark couplets, `lineSimilarity()` between neighbours finds them (+120 at ≥0.6, or ≥0.45 when they are more alike than either is to its other neighbour - Gilgamesh, Enkidu and the Netherworld; Enmerkar). Apply parity only where two lines fill a row (both ≥28% - Maqlû). A line ending with `:` is never paired with the sentence before it (OB Gilgamesh); a quotation closing inside a speech (`'`) ends its row like a speech does (SB Gilgamesh). A quotation after `;` or a dash is a name, not a speech (Gudea's lists). **Speech**: a speech's close as a cost (–25) rather than `"always"` where speeches are short or broken (Etana: `"always"` cost 6 rows). **`repeatedPassages`**: compare `"alike"`, `"similar"` and `"free"` every time. `"similar"` helped only where messages are retold with single words changed (Enmerkar). It hurt where repeats vary on purpose (creation texts, Inanna, king lists). In law codes even `"alike"` copies one law's break into a parallel law where the shared clause plays another part (Lipit-Ishtar, Hammurabi: `"free"`).
- 2026-10-27, pitfalls: a word-named exception that matches nothing is not reported - confirm that each one changed its run. Strip markup, brackets and vowel points from the line and from the name the same way (removing points also drops a yod written only by its vowel: נַנָּיָה "ננ(י)ה →"). `compare --with=` works only for a copy beside the module (`translation.rtl.layout.bak-N.ts`, deleted after). Two `"never"`s whose three lines do not fit on one row are dropped silently. A file that is GENERATED (OB Gilgamesh, by `build_translation.py`) turns OBSOLETE on every rebuild.